DHBW
Duale Hochschule
Baden-Württemberg
Ravensburg
Campus Friedrichshafen

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

Projektarbeit T3_3100

des Studiengangs **Informatik IT-Security** an der
Dualen Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

von **Leon Luca Eltrich**

Abgabedatum: **08.09.2025**

Bearbeitungszeitraum:	10 Wochen
Matrikelnummer, Kurs:	4276625, TIS23
Dualer Partner:	doubleSlash Net-Business GmbH, Friedrichshafen
Betreuer*in des Dualen Partners:	- - -
Gutachter*in der Dualen Hochschule:	Prof. Dr. Axel Hoff

Eigenständigkeitserklärung

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum	Unterschrift

Abbreviations

LLM Large Language Model

DTO Data Transfer Object

CLI Command Line Interface

I/O input/output

IPC Inter-Process Communication

OS Operating System

Inhaltsverzeichnis

1 Introduction **1**

1.1 Background and Motivation 1

1.2 The Problem 1

1.3 Proposed Solution: A Local-First Approach 2

1.4 Research Questions 2

1.5 Expected Contributions 2

2 Background **4**

3 Methodology **5**

3.1 Literature Review 5

3.2 System Design 5

3.3 User Study 5

3.4 System Evaluation 5

4 Literature Review **6**

4.1 Related Works 6

5 Systems Design And Implementation **7**

5.1 Design 7

5.1.1 Requirements 7

5.1.2 Architectural Pattern 10

5.1.3 System Context Model (Level 1) 11

5.2 Implementation 13

6 Evaluation **14**

6.1 Technical Benchmarking 14

6.2 User Study 14

7 Results **15**

8 Discussion **16**

8.1 Validity Concerns 16

8.2 Answers To The Research Questions 16

9 Conclusion And Future Work **17**

1 Introduction

1.1 Background and Motivation

For decades, the Command Line Interface (CLI) has served as a fundamental and powerful mechanism for navigating and managing computer systems. Command-line applications remain ubiquitous across software development, system administration, data analysis, and scientific computing. They offer unmatched efficiency, scriptability, and granular control over system resources.

However, despite its powerful features and long-standing existence, the command line continues to present a challenge, especially to novice users. The interface relies heavily on strict syntactical rules, cryptic abbreviations, and inconsistent command option flags across different utilities. While newcomers might struggle to interpret the highly technical descriptions found in traditional man pages or help manuals, the cognitive load affects seasoned users as well. Applications frequently possess an excessive number of options, flags, and arguments, making it difficult to utilize tools to their full potential without constant reference to external documentation.

1.2 The Problem

The traditional methods for bridging this knowledge gap, such as manual page consultation, web searches, or community forums, often require significant context switching, which disrupts the user's workflow. Recently, cloud-based Large Language Models (LLMs) such as ChatGPT or Gemini have emerged as popular alternatives for providing immediate command-line guidance.

While they offer highly intuitive natural language interaction, they suffer from several critical limitations in the context of system administration and software development:

- Privacy and Security Risks: Relying on cloud APIs requires transmitting user queries – which may contain sensitive system paths, file names, or proprietary tool names – to external servers.
- Networking Constraints: Cloud models are inherently unavailable in restricted, air-gapped, or offline environments, which are common in enterprise server management and secure development setups.
- Domain-Specific Knowledge Gap: General-purpose models often lack knowledge of internal, closed-source, or highly niche CLI applications specific to a particular organization's workflow.
- Ressource Usage: While open-source LLMs exist, they tend to be quite hardware and ressource heavy, which limits their feasibility.

1.3 Proposed Solution: A Local-First Approach

This thesis aims at addressing the challenges outlined in section 1.2 by exploring a local-first, privacy-preserving and extensible command option retrieval system. The core objective is to design, implement and evaluate a system that interprets natural-language prompts and accurately retrieves the correct command-line options without relying on external services.

By utilizing natural language processing approaches such as embeddings and lightweight langauge models, the proposed system will map user-generated descriptions of what they are trying to do (e.g. ”List all files in this directory, including the ones not shown.”) to the command-line options required for the task. Crucially, this proof-of-concept is designed to operate strictly offline, with a focus on acceptable response times and hardware usage, utilizing only standard CPU hardware.

1.4 Research Questions

To systematically evaluate the feasibility and performance of the proposed local-first approach, this thesis seeks to address several core research questions. These questions are designed to bridge the gap between technical optimization and human-centric usability, focusing specifically on the constraints of offline, CPU-bound environments.

The study is guided by the following specific research questions:

- **RQ1:** Which local NLP techniques can effectively map natural language descriptions to command-line options with limited hardware resources available?
- **RQ2:** What trade-offs exist between accuracy, resource usage and extensibility in such a system?
- **RQ3:** How can the system be extended with new commands and options with minimal overhead?

1.5 Expected Contributions

This thesis provides both a practical tool and new scientific insights into how natural language can improve command-line interactions. The primary contributions include:

First, this work delivers a functional proof-of-concept system that proves it is possible to retrieve command options locally and privately. Second, it provides a detailed analysis of the technical trade-offs – balancing accuracy, speed, and memory usage – when running NLP models on standard CPU hardware.

Additionally, the thesis introduces an extensible architecture that allows developers to easily add natural language support to other CLI tools. Finally, the research offers an understanding

of the preceived effectiveness of the tool as experienced by users. Together, these results offer a practical path toward making the command line more accessible without compromising user privacy.

2 Background

3 Methodology

3.1 Literature Review

3.2 System Design

3.3 User Study

3.4 System Evaluation

- Test catalog generation - Test Set generation

4 Literature Review

4.1 Related Works

5 Systems Design And Implementation

5.1 Design

5.1.1 Requirements

Before defining the functional behaviors and quality attributes of the system, it is necessary to establish the immutable physical and operational boundaries within which the software must execute. These environmental constraints dictate the hardware baseline for all subsequent performance evaluations and formally enforce the strict offline, zero-dependency philosophy of the proposed architecture. The fundamental constraints governing the local execution environment are detailed in Table 1.

Tabelle 1: Hardware and Environmental Constraints

ID	Constraint	Description
ENV-1	Target Hardware	All performance metrics must be achieved on a baseline Proxmox VM running a Linux based operating system, configured with 4 vCPUs (AMD Ryzen 7 5700X architecture), 8GB of 2133MHz DDR4 RAM, and a Samsung SSD 970 Evo Plus NVMe drive.
ENV-2	Offline Isolation	The system must function completely offline. It must not initiate any outbound network requests under any circumstances.
ENV-3	Zero Third-Party Daemon	The system must be entirely self-contained. While the system's own architecture may utilize background processes, it must not require the installation or execution of third-party standalone server software.
ENV-4	Headless Execution	The target environment is strictly headless. The system must not rely on or require graphical windowing systems or desktop environments for any user interaction or background execution.

With the operational boundaries established, the Functional Requirements (FR) define the explicit behaviors, interactions, and data-processing capabilities the system must exhibit. These requirements define the „what“ of the system, strictly abstracting away internal implementation details to focus entirely on user-facing features, catalog lifecycle management, and logical orchestration. The complete functional specification is presented in Table 2.

While Functional Requirements dictate what the system must do, the Non-Functional Requirements (NFR) define the standard of quality to which the system must adhere. These attributes establish the rigorous performance budgets, architectural modularity paradigms, security perimeters, and latency thresholds required to ensure the tool remains a viable, high-performance

Tabelle 2: Functional Requirements (FR)

ID	Requirement	Description
FR-1	Command Generation	The system must accept natural language queries and return a syntactically correct command-line string comprising the base tool, relevant option flags, and placeholders for user-specific positional parameters.
FR-2	Non-Execution Boundary	The system must strictly output the generated command as text and never autonomously execute it in the host shell.
FR-3	Authenticated Ingestion	The system must provide a programmatic interface to ingest and index new tool catalogs, utilizing an authentication or cryptographic authorization mechanism to prevent unauthorized manipulation.
FR-4	Catalog Lifecycle Management	The system must provide authenticated interfaces allowing authorized entities to list, update, and remove previously ingested tool catalogs.
FR-5	State Persistence	The system must persist all ingested catalogs and computed retrieval states locally, ensuring immediate availability across system reboots and terminal sessions without requiring re-ingestion.
FR-6	Syntactical Validation	The system must logically evaluate generated commands to ensure no mutually exclusive flags are recommended simultaneously, and that any required positional ordering is strictly respected.
FR-7	Interactive Disambiguation	If a query is ambiguous, triggers a flag conflict, or falls outside the domain of ingested catalogs, the system must halt and prompt the user for clarification rather than returning an invalid command.
FR-8	User Interface	The system must expose a user interface for all user-facing operations, including query submission, catalog management, and interactive disambiguation.
FR-9	Configuration Management	The system must provide an accessible mechanism for the user to view and persistently modify local system preferences without requiring direct manipulation of the underlying storage mechanism or compiled source code.

alternative to cloud-based assistants. The quantifiable metrics and structural constraints governing the system's execution quality are enumerated in Table 3.

Tabelle 3: Non-Functional Requirements (NFR)

ID	Requirement	Description
NFR-1	Query Latency	The system must return a completed command recommendation in under 1000ms from the moment the query is submitted.
NFR-2	Cold Startup Latency	The application must initialize and be ready to process input in under $1000ms$.
NFR-3	Memory Bounding	The peak RAM utilization of the system process must never exceed $1024MB$ at any time during operation.
NFR-4	Latency Scalability	The system must maintain its $< 1000ms$ query latency when the persistent state is scaled to hold definitions for up to ** Average number of CLI tools installed on a linux machine ** distinct tools.
NFR-5	Retrieval Quality	The system must achieve a minimum Macro-Averaged F1-Score of **[Target]** and a Jaccard Similarity of **[Target]** against the established evaluation dataset.
NFR-6	Storage Modularity	The system architecture must strictly decouple the core application logic from the persistent state mechanisms. It must be possible to completely replace the underlying storage backend without modifying the domain logic, retrieval algorithms, or user interfaces.
NFR-7	Search and Matching Modularity	The system architecture must isolate all specific search and matching mechanisms behind abstract interfaces.
NFR-8	Similarity and Rank Aggregation	The system must support the multi-strategy evaluation of candidate command-line options against the user's natural language query. It must provide an internal mechanism to mathematically normalize and fuse disparate similarity scores from independent matching algorithms into a single, unified ranking before outputting the finalized result.

Continued on next page ...

Table 3: Non-Functional Requirements (Continued)

ID	Requirement	Description
NFR-9	Linux Deployment	The system must be deployable as a self-contained artifact for Linux environments, requiring no pre-installed language runtimes or package managers on the host machine.
NFR-10	IPC Network Isolation	Any IPC mechanisms utilized by the system must be strictly bound to local OS constructs, explicitly denying any non-local network access.
NFR-11	State Access Control	The system must utilize host OS access control mechanisms to ensure its persistent state data is logically isolated and protected from unauthorized modification by other non-privileged users.
NFR-12	Concurrency Resilience	The system must safely process simultaneous execution requests from multiple terminal instances without causing deadlocks, process crashes, or state corruption.
NFR-13	Input Sanitization	The system must sanitize all user inputs and programmatic payloads to prevent injection attacks against the underlying persistent state mechanism.
NFR-14	Opt-In Diagnostic Logging	By default, any diagnostic logs generated by the system must be strictly scrubbed of the user's raw natural language queries. Unredacted query logging must require explicit user opt-in.

5.1.2 Architectural Pattern

The design of the system is fundamentally driven by the non-functional requirements governing latency, memory footprint, and modularity. Specifically, the system must process natural language queries in under 1000ms (NFR-1, NFR-4) within a 1024 MB memory limit (NFR-3), while ensuring that the core domain logic remains strictly decoupled from volatile input/output (I/O) operations, allowing both the persistent storage backend (NFR-6) and the specific search and matching algorithms (NFR-7) to be seamlessly replaced.

To satisfy these constraints, a traditional Layered (N-Tier) Architecture was discarded. While a layered approach neatly categorizes concerns into different tiers (e.g. presentation, business and data), it introduces a transitive top-down dependency structure [1]. As Richards and Ford assert, layers in this architecture remain isolated only if the contracts between them remain unchanged. However, in a standard layered model, the data access layer and external libraries define these contracts. Consequently, the core business logic is transitively coupled to the paradigms of

the infrastructure. If the local storage technology or the specific text-matching algorithms were fundamentally replaced, the cascading changes to the underlying contracts would inevitably force direct modifications within the business logic. This top-down coupling directly violates the system's modularity constraints (NFR-6, NFR-7).

To decouple the algorithmic core from the storage and execution components, the system relies on the Dependency Inversion Principle. While modern dependency-inverting paradigms such as Clean Architecture [2] and Onion Architecture [3] offer robust domain isolation, they prescribe concentric internal layers that mandate extensive Data Transfer Object (DTO) mapping and rigid interface boundaries. For a localized command-line utility, operating under strict computational and latency budgets (NFR-1, NFR-3, NFR-4), this enterprise-level boilerplate introduces unjustifiable computational overhead. Furthermore, it inherently increases the structural complexity of the codebase, making future maintenance and the extension of new features unnecessarily burdensome for a utility application.

Therefore, this system adopts a **Hexagonal Architecture**, also known as the Ports and Adapters pattern [4]. This pattern provides the necessary macro-level boundary control without dictating internal bureaucratic layers, allowing developers to maintain focus on the actual implementation of the core feature set. The architecture is divided into two strict conceptual zones:

- **The Core (Inside):** This encapsulates the domain entities, the command validation (FR-6), and the mathematical normalization and score aggregation logic (NFR-8). The Core possesses zero knowledge of the external world, it dictates its storage and matching needs purely through abstract interfaces (Ports).
- **The Infrastructure (Outside):** This encompasses all external adapters that fulfill the Core's interfaces. This includes the user-interface, the storage backend, as well as the specific execution engines for the command option evaluation.

By implementing interface ports at the absolute boundary of the Core, the external mechanisms – whether they are the user interface, storage backend, or the command option evaluation mechanisms – are forced to depend on the Core rather than the inverse. This guarantees that the core natural language orchestration and rank aggregation math can be compiled, tested, and executed entirely independently of the chosen storage backend or matching technologies, successfully fulfilling all modularity constraints.

5.1.3 System Context Model (Level 1)

To formalize the operational boundaries of the software and verify the isolation constraints, a Level 1 System Context diagram was modeled using the C4 standard **brown2018c4**. As illustrated in Figure 1, the proposed tool is treated as a unified, opaque system operating entirely within a headless Linux environment (ENV-4). This high-level abstraction defines the absolute

perimeter of the application, explicitly identifying the external actors and systems permitted to interact with it.

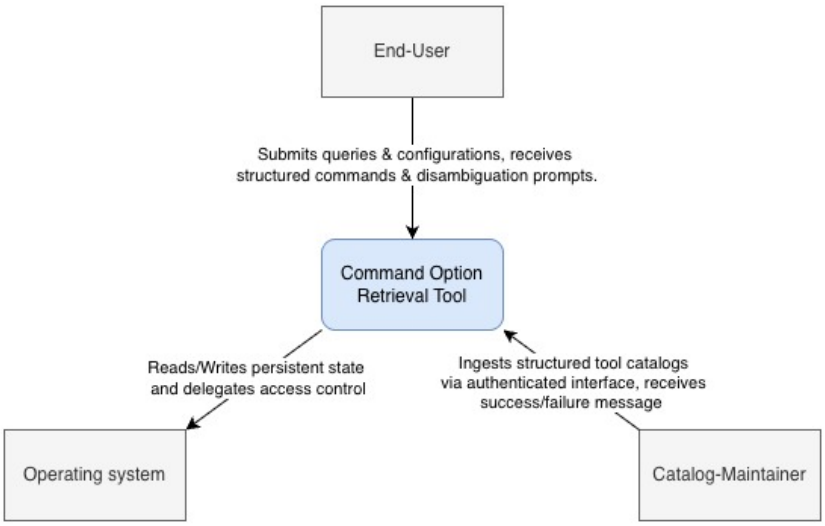

Abbildung 1: Level 1 System Context Model

The system interacts with two primary driving actors that initiate operations:

- **The End-User:** Interacts with the system by submitting natural language queries and configuration parameters (FR-1, FR-9). In turn the End-User receives the command best fitting their query and disambiguation prompts (FR-7, FR-9).
- **The Catalog-Maintainer:** Represents either a human developer or an application developed by them. This actor interfaces with the system via an authenticated programmatic boundary to ingest new or updated, structured tool catalogs, receiving synchronous validation receipts upon success or failure (FR-3). The Catalog-Maintainer also has the ability to delete their catalogs usign the authenticated interface.

Furthermore, the system relies on a single driven external system: the **Host Operating System (OS)**. The application initiates outward requests to the host operating system to read and write its persistent state. While the system manages application-level authentication, it defers to the underlying OS file permission mechanisms to enforce OS-level access control for all persisted data (NFR-11).

Most critically, the Context Model serves as visual and structural proof of the system's privacy and offline execution limitations (ENV-2). It also demonstrates that the zero third-party daemon reliance (ENV-3) is considered. The defined perimeter strictly excludes any outbound network or third party daemon dependencies. By restricting the attack surface and data flow strictly to

the local host operating system and authorized standard inputs, the architecture guarantees that all user queries and proprietary tool descriptions remain entirely localized.

5.2 Implementation

- Rust

Beschreibung der Module und wie sie ineinandergreifen (Diagram?)

Ggf. eine beschreibung wie man besonders effectiv die Beschreibungen für das Embedding formuliert.

tar, mv, cp,

6 Evaluation

6.1 Technical Benchmarking

6.2 User Study

7 Results

8 Discussion

8.1 Validity Concerns

8.2 Answers To The Research Questions

9 Conclusion And Future Work

Literaturverzeichnis

[1] M. Richards und N. Ford, *Fundamentals of software architecture: an engineering approach*. O'Reilly Media, 2020.

[2] R. C. Martin, *Clean architecture: a craftsman's guide to software structure and design*. Prentice Hall Press, 2017.

[3] J. Palermo. „The Onion Architecture". Adresse: `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/`.

[4] A. Cockburn. „Hexagonal Architecture". Adresse: `https://alistair.cockburn.us/hexagonal-architecture`.

Abbildungsverzeichnis

1 Level 1 System Context Model 12

Tabellenverzeichnis

1 Hardware and Environmental Constraints 7
2 Functional Requirements (FR) 8
3 Non-Functional Requirements (NFR) 9
4 Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung 21

KI-Verzeichnis

Werkzeug	**Beschreibung der Nutzung**
ChatGPT	Der Einsatz erfolgte in der gesamten Arbeit zur sprachlichen Gestaltung. Konkret diente es dazu, vorhandene Texte stilistisch an den wissenschaftlichen Schreibstil anzupassen und präzise Formulierungen zu entwickeln. Außerdem wurde wie in Kapitel **??** bzw. Kapitel **??** beschrieben, die Funktion *ChatGPT Deep Research* für das Identifizieren relevanter Quellen genutzt. Limitationen wurden in besagten Kapiteln reflektiert. Darüber hinaus kam ChatGPT zur Klärung fachlicher Verständnisfragen im Verlauf der Recherche und Umsetzung des Projektes zum Einsatz.
Perplexity	Der Einsatz erfolgte wie in den Kapiteln **??** und **??** beschrieben, um relevante Quellen zu identifizieren. Wie bei *ChatGPT Deep Research*, wurde der Einsatz in den Kapiteln kritisch hinterfragt und Limitationen identifiziert.

Tabelle 4: Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung